



Jyväskylän yliopisto

KYBS2004EN26K2 - Penetration Testing Report

by Riki Järvinen

Start of testing: March 16, 2026

End of testing: May 24, 2026

Contents

1	Executive Summary	1
2	Vulnerability overview	2
3	Results	3
3.1	Subdomain 1	3
3.1.1	Password cracking with John the Ripper and Hashcat	3
3.1.1.1	Minimal proof of concept	3
3.1.1.2	Verdict	4
3.2	Subdomain 2	5
3.2.1	RSA key-cracking	5
3.2.1.1	Minimal proof of concept	5
3.3	Subdomain 3	7
3.3.1	RSA file-message decryption	7
3.3.1.1	Minimal proof of concept	7
3.4	Subdomain 4	8
3.4.1	RSA file-message forged signature	8
3.4.1.1	Minimal proof of concept	8
4	Appendices	11
4.1	Appendix #1	11
4.2	Appendix #2	11

1 Executive Summary

This report summarizes the findings of KYBS2004EN26K2 courses given exercises. Exercises were performed in close environment using platform TIM. The report is structured in a way that can give reader a clear overview of the findings and their implications.
<add more>

2 Vulnerability overview

#	Vulnerability	Summary	Risk	Section
1	XSS (Reflected)	XSS vulnerability allowing arbitrary script injection.	Medium	3.1.1
2	Command Injection	Vulnerability found that attacker could execute arbitrary commands on text field.	High	3.1.2

3 Results

This part of the document contains all the necessary information about the cracking done in given exercises. The cracking exercises are organized by domains, with each domain representing a specific testing environment. Each issue is described in detail, including its basic information, minimal proof of concept.

3.1 Subdomain 1

Platform: TIM environment

Server IP address: 193.167.189.112

3.1.1 Password cracking with John the Ripper and Hashcat

In this exercise, i performed password cracking utilizing two popular tools: John the Ripper and Hashcat. The goal was to identify weak passwords and usernames that could potentially be exploited by "attackers".

Usernames and passwords were obtained from the TIM environment, and both tools were tested to perform the cracking process.

Usernames found: anna, riku Passwords found: airborne, columbia3

3.1.1.1 Minimal proof of concept

General step: I started the exercise by figuring out where the passwords were located. It was found at

```
/usr/share/common_3k.txt
```

John the Ripper:

I used unshadowing technique to prepare the password file for cracking, called the file "myfile". The command i used was:

```
unshadow /etc/passwd.kybs2004 /etc/shadow.kybs2004 > myfile
```

where passwd and shadow files were located at /etc directory.

With this i could start the cracking process with John the Ripper.

For the task i had to figure out from the rules what rules i wanted to use. For the first cracking process i used no flags with the command:

```
john --wordlist=/usr/share/common_3k.txt myfile
```

which resulted in finding the password "airborne" for user "anna".

```
rimijarv@attack_tools:~$ john --wordlist=/usr/share/common_3k.txt myfile
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Press 'q' or Ctrl-C to abort, almost any other key for status
airborne          (anna)
```

For the second cracking process i used the flag "-rules" which added some rules to the cracking process. In particular we wanted to use [0-9] rule which adds digits to the end of the password. The command i used was:

```
john --wordlist=/usr/share/common_3k.txt --rules myfile
```

which resulted in finding the password "columbia3" for user "riku".

```
rimijarv@attack_tools:~$ john --wordlist=/usr/share/common_3k.txt --rules myfile
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Remaining 2 password hashes with 2 different salts
Press 'q' or Ctrl-C to abort, almost any other key for status
columbia3        (riku)
```

Hashcat:

Hashcat was used to perform password cracking even though i had already found them with JTR. With hashcat i had to figure out what hash type was used for password to determine the correct flag. I found out that the hash type was "sha512crypt" which corresponds to flag "-m 7400". In attack mode i figured that attack mode 'Straight' corresponds to flag "-a 0", which means basic attack. With all this information i could form the command for hashcat:

```
hashcat -m 7400 -a 0 myfile /usr/share/common_3k.txt -r /usr/share/hashcat/rules/bes
```

```
rimijarv@attack_tools:~$ hashcat -m 7400 -a 0 myfile /usr/share/common_3k.txt -r /usr/s
-force
hashcat (v5.1.0) starting...
```

which resulted in finding the same passwords as with JTR, "airborne" and "columbia3".

```
$5$tpyMNTGQ1cOgGOAF$j5GgFAkXYlqUoVZsehNwebvXSY7KkA/lhuGNWFTXxi3:columbia3
$5$euDnWoHEdDFn9Htn$Mfyqr2ccdEjHbUbX.WmUXDYlqLROqXTgJEMAJloZ.n8:airborne
```

3.1.1.2 Verdict

Overall both tools were effective in cracking the passwords, with hashcat being slightly faster in cracking the passwords. JTR cause some headache to figure out what to do but once i got the hang of it, it was pretty straightforward to use. That also helped with understanding how hashcat works.

3.2 Subdomain 2

Platform: TIM environment

Server IP address: 193.167.189.112

3.2.1 RSA key-cracking

In this exercise i was tasked to crack an RSA key pair, which is a common cryptographic method used for secure communication. The goal was to identify the "factors" associated with the given public key file.

Exercise was performed in private environment using TIM platform, and the key pair was provided in a file named "pub.pem".

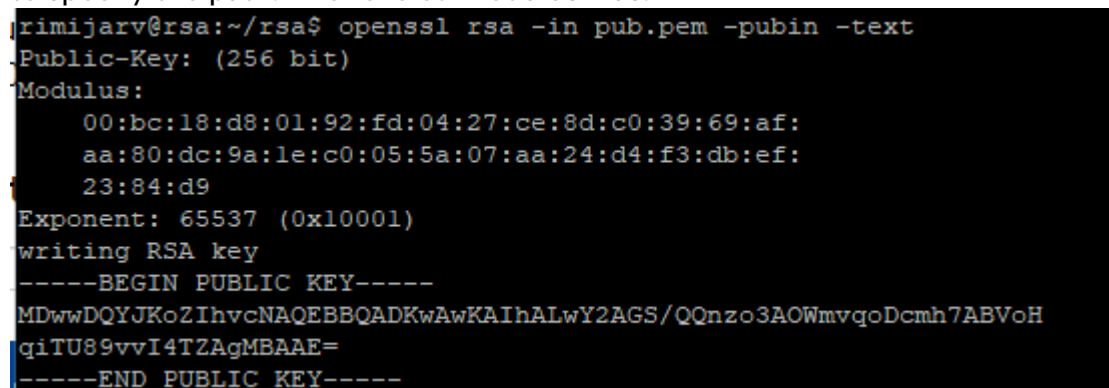
3.2.1.1 Minimal proof of concept

Firstly locate the pem file, in this case pub.pem file was located in /home/rimijarv/rsa/ directory.

Public key was coded in base64, for this we need to use openssl rsa to reveal the modulus. Openssl rsa has a flag "-pubin" which indicates that the input file has the public key. To select the file we use flag "-in". Lastly for human readable output we need to use flag "-text". With all this information we can form the command:

```
openssl rsa -in pub.pem -pubin -text
```

pub.pem was located in the same directory as the command was executed, so no need to specify the path. The reveled modulus was:



```
rimijarv@rsa:~/rsa$ openssl rsa -in pub.pem -pubin -text
Public-Key: (256 bit)
Modulus:
 00:bc:18:d8:01:92:fd:04:27:ce:8d:c0:39:69:af:
 aa:80:dc:9a:1e:c0:05:5a:07:aa:24:d4:f3:db:ef:
 23:84:d9
Exponent: 65537 (0x10001)
writing RSA key
-----BEGIN PUBLIC KEY-----
MDwwDQYJKoZIhvcNAQEBBQADKwAwKAIhALwY2AGS/QQnzo3AOWmvqoDcmh7ABVoH
qiTU89vvI4TZAgMBAAE=
-----END PUBLIC KEY-----
```

With the revealed information we need to convert modulus (hex) to base10. With this we need to understand that modulus was in hexadecimal format, which means that it is base16. Remember to remove ":". To convert it to base10 we can use python:

```
python3 -c "print(int('00bc18d80192fd0427ce8dc03969afaa80dc9a1ec0055a07aa24d4f3dbef2
```

which resulted in the following base10 number:

```
85078710682859295091507409553095548352098481513760243430344420876980368671961
```

```
rimijarv@rsa:~/rsa$ python3 -c "print(int('00bc18d80192fd0427ce8dc03969afaa80dc9a1ec0055a07aa24d4f3dbef216))"
85078710682859295091507409553095548352098481513760243430344420876980368671961
```

With the base10 number we needed to figure out what the factors were. For this we were given an tool called msieve. Msieve was extremely easy to use where you just needed to:

```
msieve -v 85078710682859295091507409553095548352098481513760243430344420876980368671
```

Parameter being the base10 number we got from the previous step. which resulted in the following factors:

```
p39 factor: 256685714581489668977306825067425055347
```

```
p39 factor: 331450898315766891287300607911209823363
```

```
p39 factor: 256685714581489668977306825067425055347
p39 factor: 331450898315766891287300607911209823363
elapsed time 00:01:41
```

3.3 Subdomain 3

Platform: TIM environment

Server IP address: 193.167.189.112

3.3.1 RSA file-message decryption

In this exercise i was tasked to decrypt a message and return that message. Task was continous from the previous exercise, which results i could use to help decrypting the message. The end goal was to identify the decrypted message.

Task was performed in private environment using TIM platform, and the encrypted message was provided in a file named "encrypted_flag.rsa".

3.3.1.1 Minimal proof of concept

To construct the private key used to decrypt the message, i needed to figure out what the python script did. The script was used to generate the private key and i could utilize the factors obtained from the previous exercise as the 'pub.pem' file was the same. I also needed to save the private key into a .pem file so i can use it later. Filed was called "key.pem". With this information i could run the script with given parameters

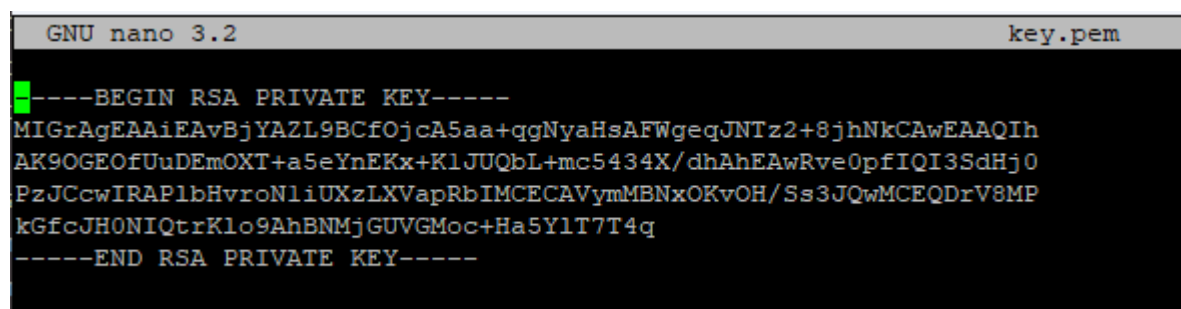
```
python3 reconstruct_privkey.py <pem file> <P factor> <Q factor>
```

```
P factor: 256685714581489668977306825067425055347
```

```
Q factor: 331450898315766891287300607911209823363
```

```
pem file: pub.pem
```

```
python3 reconstruct_privkey.py pub.pem 256685714581489668977306825067425055347  
331450898315766891287300607911209823363 > key.pem
```



```
GNU nano 3.2 key.pem  
-----BEGIN RSA PRIVATE KEY-----  
MIGrAgEAAiEAvBjYAZL9BCfOjcA5aa+qgNyaHsAFWgeqJNTz2+8jhNkCAwEAAQIh  
AK9OGEOfUuDEmOXT+a5eYnEKx+KlJUQbL+mc5434X/dhAhEAWRve0pfIQI3SdHj0  
PzJCcwIRAPlbHvroNliUXzLXVapRbIMCECAVymMBNxOKvOH/Ss3JQwMCEQDrV8MP  
kGfcJH0NIQtrKlo9AhBNMjGUVGMoc+Ha5YlT7T4q  
-----END RSA PRIVATE KEY-----
```

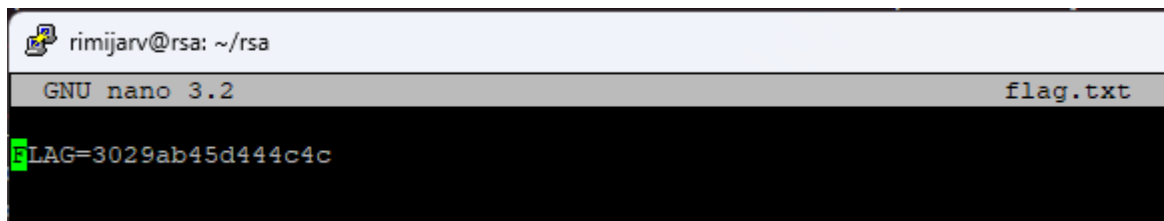
With this given constructed private key, i could use openssl to decrypt the message. For this i needed to use openssl rsautl and i needed to specify the correct flags for the command. -Help gave alot of useful information about the flags which i managed to utilize to form the command:

- decrypt: indicates that the operation to be performed is decryption.
- in: specifies the input file containing the message to be decrypted (encrypted_flag)
- inkey: specifies the file containing the private key to be used for decryption (key)
- out: specifies the output file where the decrypted message will be saved.

```
openssl rsautl -decrypt -in encrypted_flag.rsa -inkey key.pem -out flag.txt
```

which resulted in the decrypted message being saved in "flag.txt" file. The found flag was:

```
FLAG=3029ab45d444c4c
```



3.4 Subdomain 4

Platform: TIM environment

Server IP address: 193.167.189.112

3.4.1 RSA file-message forged signature

In this exercise i was tasked to forge a digital signature from given private key. Task was continous from the previous exercises (2, 3), which results i could use to solve the given exercise. Task utilizes all the learned knowledge from the previous exercises.

Task was performed in private environment using TIM platform.

3.4.1.1 Minimal proof of concept

To start the exercise i needed to create a private key file, which i had done before in the previous exercise.

```
python3 reconstruct_privkey.py <pem file> <P factor> <Q factor>
python3 reconstruct_privkey.py pub.pem 256685714581489668977306825067425055347 33145
```

With the private key created, i can start creating a digital signature. For doing this i need to use openssl rsautl again, but this time with different flags.

- sign: indicates that the operation to be performed is signing.
- in: specifies the input file containing the message to be signed (SIGN_THIS.txt).
- inkey: specifies the file containing the private key to be used for signing (key.pem).
- out: specifies the output file where the generated signature will be saved (signature.dat).

```
openssl rsautl -sign -in SIGN_THIS.txt -inkey key.pem -out signature.dat
```

To validate that the signature was forged i needed to check the signature file with cat and see if it had content. Alternatively i could have used some hexdump tool to check the content.

To verify the signature, i used openssl rsautl again and had to figure out the correct flags for this. The public key file was already given in the exercise so there was no need to create it.

- verify: indicates that the operation to be performed is signature verification.
- in: specifies the input file containing the signature to be verified (signature.dat).
- inkey: specifies the file containing the public key to be used for verification (pub.pem).
- pubin: indicates that the input key is a public key.

```
openssl rsautl -verify -pubin -inkey pub.pem -in signature.dat
```

Which resulted in: SIGN=6bd6e0dd002388d

To return the signature in the correct format, i had to figure out how to do it with rsautl.

With the given help i managed to figure out the following command:

- sign indicates that the operation to be performed is signing.
- inkey specifies the file containing the private key to be used for signing (private_key.pem).
- in specifies the input file containing the message to be signed (SIGN_THIS.txt).
- hexdump indicates that the output should be displayed in hexadecimal format.
- > flag.txt to specify where the output should be saved.

```
openssl rsautl -sign -inkey private_key.pem -in SIGN_THIS.txt -hexdump > flag.txt
```

With this i can see the hexadump of the signature and give the result. The found signature was:

```
0000 - 75 c3 57 d1 93 f5 53 16-c7 65 fc 51 cf 1d b4 7d    u.W...S..e.Q...
0010 - 61 e5 37 a8 4c 6c a9 52-b6 e8 41 51 38 a9 1e d8    a.7.Ll.R..AQ8...
```

```
rimiJarv@rsa:~/rsa$ cat flag.txt
0000 - 75 c3 57 d1 93 f5 53 16-c7 65 fc 51 cf 1d b4 7d    u.W...S..e.Q...}
0010 - 61 e5 37 a8 4c 6c a9 52-b6 e8 41 51 38 a9 1e d8    a.7.Ll.R..AQ8...
```

And with this i could form the end result for the exercise: 75 c3 57 d1 93 f5 53 16-c7 65 fc 51 cf 1d b4 7d 61 e5 37 a8 4c 6c a9 52-b6 e8 41 51 38 a9 1e d8

4 Appendices

4.1 Appendix #1

Appendix 1.

4.2 Appendix #2

Appendix 2.